아키텍쳐 정리

GPT-OSS-20B

- Chain-of-Thought(CoT) 제공 (지도 학습 기반 X)
- low, medium, high로 추론 수준 조절 가능

이미지 처리 중심

Qwen2.5-VL-7B-Instruct

- 이미지 처리에 좋고 Instruction을 잘 따른다.

Reference

1. **SocratiQ: A Generative AI-Powered Learning Companion for Personalized Education and Broader Accessibility**
   - https://arxiv.org/pdf/2502.00341
   - **Personalized Explations**
     - 사용자의 Level에 따라 다른 System prompt를 사용
   - **Adaptive assessments**
     - 하나의 ML Text book에 대해 Quiz 버튼을 누르면 그 부분을 Prompt 속에 직접 넣어 관련된 Question 3개를 만들어 달라고 요청
     - **RAG를 통해 얻은 Vector를 Prompt에 넣는 것과 비슷**

Quiz Generation Prompt:

```
Create a quiz from a CHAPTER SECTION.
The quiz should have 3 questions in JSON format:
- Q1 \& Q2: Directly related to the quote's content.
- Q3: Requires deeper understanding.
Use this JSON template, modifying it as needed:
{"questions": [
    {"question": "Q1 here?",
      "answers": [
        {"text": "A1", "correct": true/false,
        "explanation": "explanation"},
        {"text": "A2", "correct": false,
        "explanation": "explanation"},
        {"text": "A3", "correct": false,
        "explanation": "explanation"},
      ]},
    {"question": "Q2 here?", "answers": [/* options */]},
    {"question": "Q3 here?", "answers": [/* options */]}
  ]}
QUOTE: ${<insert quote here>}
CHAPTER SECTION ${<insert name and section number of chapter>}$
```

2. **Training Verifiers to Solve Math Word Problems**

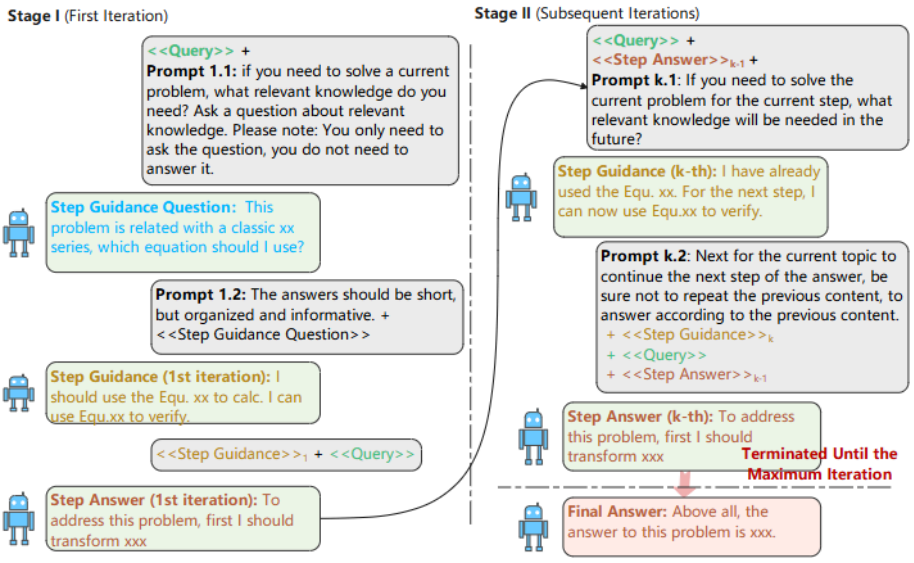
   https://arxiv.org/pdf/2110.14168

   - LLM 자체를 Fine-tuning하는 것은 GPT와 같은 Auto-regressive model의 Task가 다음 Token을 예측하는 작업만 수행하기 때문에 구조적으로 자신이 생성한 Math problem이 실제로 맞는지 검토할 수가 없다.
   - Generator LLM과 Verifier LLM 두 LLM을 사용
   - Verifier LLM은 Math dataset으로 Fine-tuning하여 문제 생성 LLM이 생성한 문제가 올바른지 확인
     - High temperture 사용
   - Verifier LLM은 Generator가 생성한 100개의 문제를 각각 평가
     - 각각의 Score를 매기도록 설정
     - 이를 위해 LLM에 스칼라 헤드(Scalar Head)를 추가
   - **추론 과정**에서는 생성기 LLM과 Verifier LLM이 모두 사용하고 Verifier에서 가장 높은 값을 얻은 문제를 사용

3. **Step Guided Reasoning: Improving Mathematical Reasoning using Guidance Generation and Step Reasoning**

https://arxiv.org/html/2410.19817v2

- Culmulative reasoning
  - 복잡한 문제는 직전 사고 과정만으로 풀리지 않고 초기의 사고 과정까지 사용해야 문제 풀이가 가능하다.
  - DAG를 사용하는 추론 방식
- Step Back Prompting (SBP)
  - Step 1에서는 문제의 본질과 문제에서 사용할만한 공식을 정리한다.
  - 원래의 Question과 Step1의 결과를 추가하여 Step2에서 Reasoning을 하도록 한다.
- Step Guided Reasoning (SGR)



  - 최대 K개의 Step으로 이루어진다.
  - 각 Step은 Step1과 Step2로 나뉜다.
    - Step1: 초기 쿼리에 대해 모델은 먼저 Step Guidance Question을 스스로 만들고, 이 질문에 답하여 첫 번째 step guidance을 생성합니다. 이 지침이 첫 step answer
      - Step2: 원래 Query에 Step answer를 추가하여 Step Guidance Question을 스스로 만들고, Step1에서의 Step answer을 Prompt에 추가하여 Step2의 Answer을 생성하도록 한다.

- 생성되는 Token 수가 많다는 단점이 존재

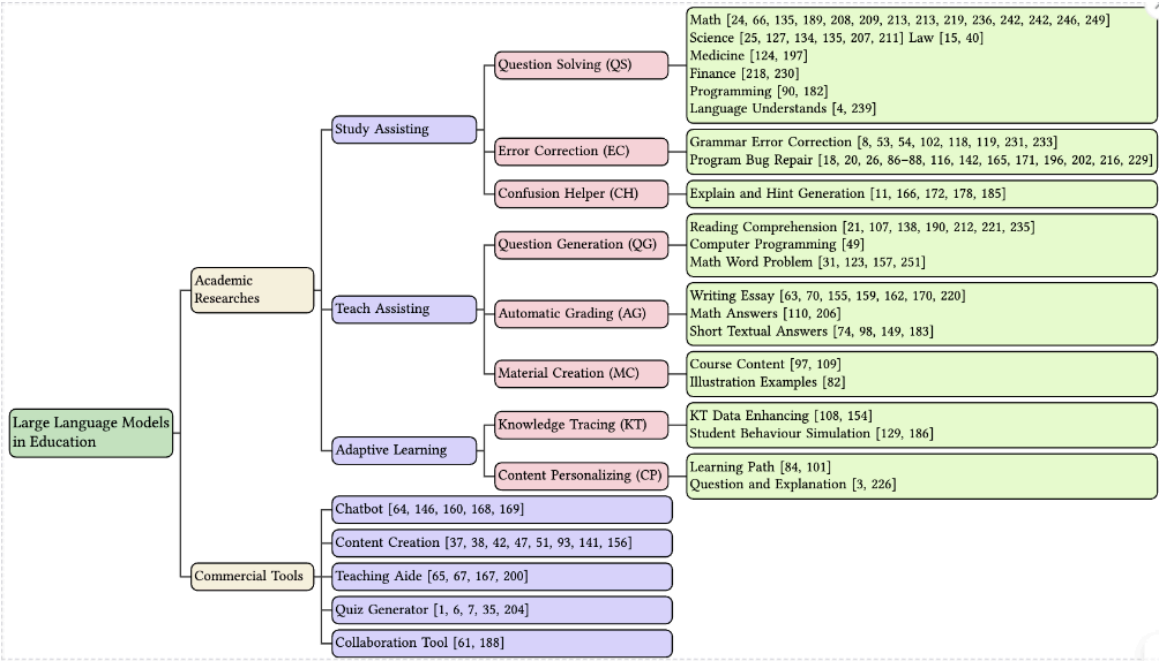
위 세 가지 방법 모두 Fine-Tuning 없이 Prompt engineering만으로도 Inference 성능을 크게 향상시켰다.

- 위 논문에서 사용하는 방법은 **SGR**이다.
- 각 Step의 max token 수는 200 ~ 500 사이

4. **Large Language Models for Education: A Survey and Outlook**

https://arxiv.org/pdf/2403.18105

LLM for education의 분류



- Question Generation
  1. 교과서, 문제를 Fine-tuning하는 방법
  2. Prompt에 생성한 문제를 검토하는 모듈 (Prompt)를 추가
  3. 문제 유형을 분류화
  4. 수학 방정식 자체를 제어
- Question Solver

1. CoT Prompting
2. Few-shot prompting
3. Tool을 활용하도록 함
4. Model이 생성한 답을 중간마다 Verifier가 검증
5. Multi Agent (LLM)이 대화를 하며 문제를 해결

5. https://medtalk.tistory.com/entry/마크다운의-장점-RAG-LLM에서-텍스트-추출과-임베딩의-용이성
   - Markdown 형식의 파일리 구조화되어있기 때문에, RAG에 유용하다.

---

1. **하나의 모델로 문제 - 해설 쌍 생성**
   a. 문제 - 해결 Data 쌍으로 Fine-tuning하여 문제 생성 + 해결 동시 학습
      i. RAG를 이용하여 문제 생성 시 참고, 문제 해설 시 교육 과정의 목표에 따른 해설 작성 유도
      ii. 이때, 문제 생성용 Fine-tuning, 문제 해결용 Fine-tuning으로 두 번 Fine-tuning하면 모델 성능이 오히려 저하
      iii. 두 가지 종류의 데이터를 **하나의 데이터셋으로 합쳐서 한 번만 Fine-tuning하면 된다**.
         - `Multi-task Fine-tuning`, `Instruction Tuning` 이라고 부른다.

```
// train.jsonl

// --- 문제 생성용 데이터 ---
{"instruction": "중학교 1학년 '소인수분해' 단원에서 난이도 '중' 문제를 생성해줘.", "output": "두 자연수 72와 120의 최대공약수와 최소공배수의 합을 구하시오."}
{"instruction": "고등학교 수학(상) '이차함수' 단원에서 난이도 '최상' 문제를 생성해줘.", "output": "이차함수 y = x² - 2ax + a² - 1의 그래프가 x축과 만나는 두 점 사이의 거리가 4일 때, 모든 실수 a의 값의 곱을 구하시오."}
```

```
// --- 문제 해결용 데이터 ---
{"instruction": "두 자연수 72와 120의 최대공약수와 최소공배수의 합을 구하시오.", "output": "1. 72를 소인수분해하면 2³ × 3² 입니다. \n2. 120을 소인수분해하면 2³ × 3 × 5 입니다. \n3. 최대공약수(GCD)는 공통 소인수의 더 낮은 지수를 택하므로 2³ × 3 = 24 입니다. \n4. 최소공배수(LCM)는 모든 소인수의 더 높은 지수를 택하므로 2³ × 3² × 5 = 360 입니다. \n5. 따라서 두 수의 합은 24 + 360 = 384 입니다."}
{"instruction": "이차함수 y = x² - 2ax + a² - 1의 그래프가 x축과 만나는 두 점 사이의 거리가 4일 때, 모든 실수 a의 값의 곱을 구하시오.", "output": "..."}
```

2. **Generator - Verifer - Solver 구조**
   a. Generator는 최상, 상, 중, 하 Tag가 정확히 붙은 문제를 Data로 이용
      i. 교육과정 + 난이도가 명시되었을 때, 해당 데이터가 나오도록 유도
   b. Verifer는 교육과정만 RAG로 사용해서 교육과정을 따르고, 문제 자체의 계산 등의 오류가 없는지를 탐색
      i. Prompt engineering으로 해결
   c. Solver는 AI hub의 데이터 셋을 이용하여 문제에 맞게 자세한 해설을 작성하도록 유도
      i. Chain-of-Thought Prompting으로 사고 과정을 출력하도록 할 수도 있음
      ii. Step Guided Reasoning을 이용하여 2-Step으로 더 면밀하게 해설을 작성하도록 할 수도 있음

GPT-OSS-20B

- System prompt를 통해 COT 과정 숨기라고 명령해도 Prompt가 길어지면 COT 과정을 숨기지 못 했다.
- SVG 능력은 비교적 떨어짐
  - SVG 코드가 필요한 도형 문제에서 생성 Token 수가 급증

Qwen3-VL-8B-Instruct

- COT 과정 숨기는 데 능숙
- 문제 퀄리티가 비교적 떨어짐

Example 1: 초등 1학년 1학기 나눗셈

```
Let's produce output.assistantfinalProblem Type: multiple choice
Problem: 서준이는 사과 34개를 가지고 있어요. 친구에게 사과 12개를 주었어요. 이제 서준이에게 사과가 몇 개 남았나요? (난이도: 하)
Choices: 1. 22, 2. 23, 3. 21, 4. 24, 5. 20
Image: N/A
```

```
Qwen3-VL-7B Instruct Output
Problem Type: multiple choice
Problem: 학생이 모형 시계를 보고 "3시 15분 전"을 읽었을 때, 시계의 시침과 분침이 가리키는 시간은 몇 시 몇 분인가요? (난이도: 하)
Choices: 1. 2시 45분, 2. 2시 30분, 3. 2시 15분, 4. 3시 15분, 5. 3시 30분
Image: <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <circle cx="100" cy="100" r="90" stroke="black" stroke-width="2" fill="none"/>
  <line x1="100" y1="100" x2="100" y2="10" stroke="black" stroke-width="3"/>
  <line x1="100" y1="100" x2="190" y2="100" stroke="black" stroke-width="3"/>
  <text x="110" y="70" font-size="14" fill="black">3시 15분 전</text>
</svg>
```

고려 사항

- User input을 Parsing하는 방법의 안정성 부여 필요
- 현재 방법은 출제 문제의 단원에 따라 난이도를 모델이 스스로 정하는 방식인데 사용자가 지정해주어야 할지
- 문제 해설 생성을 Training하고 그 모델 기반으로 연결할지